M2: из данных делаем каталог API

Вход: миграция применена, seed загружен. Сначала согласуем шесть операций.

Чтение · публично

GET /brands
GET /products
GET /products/{id}

M2: эти маршруты уже подключаем.

Изменение · только администратор

POST /products
PUT /products/{id}
DELETE /products/{id}

M2: готовим сервисы. M3: подключаем с проверкой роли.

↳ До реализации авторизации запись не открываем. Финальная приёмка CRUD завершается в M3.

Шаг 1. Прочитать список и карточку

Роутер принимает запрос, сервис делает выборку, схема формирует ответ.

Сервис · фрагмент чтения — Учебный пример

```
query = select(Product).options(
    selectinload(Product.sizes)
).order_by(Product.id)
products = list(db.scalars(query))
```

Запрос	Проверенный результат
GET /brands	North, City
GET /products	ID: 10, 20, 30
GET /products/10	North Runner + два размера
GET /products/999	404 · карточки нет

response_model=ProductRead · Decimal в JSON представлен строкой.

↳ Пустой список — нормальный ответ 200. Неизвестная отдельная карточка — 404.

Шаг 2. Проверить входные поля товара

Схема проверяет значения. Существование бренда проверяет сервис через БД.

ProductWrite · фрагмент Pydantic — Учебный пример

```
model_config = ConfigDict(extra="forbid")
name: str = Field(min_length=1,
                  max_length=120)
brand_id: int = Field(gt=0)
price: Decimal = Field(gt=0,
    max_digits=10, decimal_places=2)
# validator: strip имени, запрет пустого
```

422 · вход не подходит

price = −1 или 1.001
name = пробелы
Лишнее поле is_admin

409 · ссылка не существует

brand_id = 999 — допустимое число,
но такого бренда нет в БД.

↳ Ограничения Pydantic и PostgreSQL дополняют друг друга. Одних аннотаций Python недостаточно.

Шаг 3. Соединить фильтры через AND

Каждый параметр сужает ту же выборку, а не заменяет предыдущий фильтр.

SQLAlchemy · фрагмент — Учебный пример

```
if brand_id is not None:
    query = query.where(
        Product.brand_id == brand_id)
if max_price is not None:
    query = query.where(
        Product.price <= max_price)
```

Запрос к /products	ID в ответе
brand_id=1	10, 30
min_price=6000&max_price=8000	10
brand_id=1&max_price=8000	10

Границы

Цена включительно.
min_price > max_price → 422.

↳ Проверяем каждый фильтр отдельно, затем сочетания. Запрос возвращает только пересечение условий.

Шаг 4. Размер и поиск без повторов

Ищем товар, у которого существует нужный доступный размер.

EXISTS через relationship.any — Учебный пример

```
query = query.where(Product.sizes.any(
    (ProductSize.size == size) &
    (ProductSize.stock > 0)
))
query = query.where(Product.name.icontains(
    search.strip(), autoescape=True
))
```

Выбранное правило наличия

size=42 → товары 10 и 30.
size=43 → [], потому что stock=0.

Поиск по названию

search=nOrTh → 10 и 30.
% и _ считаем буквальными символами.

↳ EXISTS отвечает «есть ли совпадение» и не размножает товар из-за нескольких строк размеров.

Шаг 5. Страницы требуют порядка

Сначала ORDER BY, затем LIMIT и OFFSET. Показываем на неизменном каталоге.

01
Порядок
ID: 10, 20, 30

→

02
Страница 1
limit=1 · offset=0 → 10

→

03
Страница 2
limit=1 · offset=1 → 20

Завершение SQLAlchemy-запроса — Учебный пример

```
query.order_by(Product.id)
     .limit(limit).offset(offset)
```

Контракт параметров

limit: 1…100, по умолчанию 20.
offset ⩾ 0; за концом списка → [].

↳ Проверяем границы и две соседние страницы. Без ORDER BY порядок строк не обещан.

Шаг 6. Подготовить создание и PUT

Бизнес-функции уже можно проверить. HTTP-запись подключим после require_admin.

Псевдокод сервиса записи — Учебный пример

```
получить валидированную ProductWrite
найти бренд → иначе 409
для PUT найти товар → иначе 404
создать / заменить поля карточки
commit → вернуть ProductRead
ошибка записи → rollback
```

POST · новая карточка

201 + выданный сервером ID.
Размеры пока пустые: sizes=[].

PUT · редактируемые поля

name, brand_id, price обязательны.
description и image_url имеют defaults.
Размеры остаются прежними.

↳ PUT здесь заменяет поля карточки по контракту; частичное изменение PATCH не реализовано.

Шаг 7. Удалять, сохраняя историю

У товара могут быть размеры, а на размеры уже могут ссылаться другие таблицы.

Условия DELETE	Ответ
Карточки нет	404
Нет ссылок из корзины и заказов	204 · тело пустое
На размер есть ссылка	409 · данные сохранены

Проверенный опыт

Заказ 501 → размер 101 → товар 10.
DELETE /products/10 → 409.

После отказа

Товар, размеры и позиция заказа остались в исходном состоянии.

↳ Удаление размеров и товара — одна транзакция. FK и rollback сохраняют связанные данные.

Проверяем каталог через Swagger

Выберите запрос и сравните точные ID с результатом на исходном seed.

Фильтр	ID
brand_id=1	10, 30
min_price=6000&max_price=8000	10
size=43	[]
brand_id=1&size=42&max_price=8000	10
limit=1&offset=1	20

↳ 200 — только часть проверки. Сверяем содержимое, отсутствие повторов и сохранность каталога.

Ревью после M2: что уже доказано

Перед входом и ролями проверяем каталог отдельным проходом.

Проверяем сейчас

GET, фильтры, поиск, страницы.
Валидацию, ссылки, rollback.
Сервисы создания и изменения.

Открытый пункт M3

Подключить запись через require_admin.
Проверить все три метода четырьмя типами доступа.

Что приложить к ревью

Файлы + команды + ответы + состояние БД.
Отдельно — что ещё не запускали.

↳ M2 подготовлен. Требование «CRUD только админу» принимаем после реальной матрицы доступа M3.

M3: договариваемся о входе

Один формат логина и один способ передавать токен во все защищённые запросы.

Маршрут	Вход → результат
POST /auth/register	JSON email/password → 201 UserRead
POST /auth/login	JSON email/password → JWT
GET /auth/me	Bearer JWT → текущий пользователь

Подготовить M3 · из корня проекта — Учебный пример

```
python -m pip install -r requirements-m3.txt
python setup_jwt.py
python -m uvicorn app.store_main:app
# По умолчанию порт 8000
```

Swagger

Логин вызываем вручную.
В Authorize вставляем access_token.

↳ В этой практике login принимает JSON. Для запросов используется HTTPBearer.

Шаг 1. Регистрация обычного пользователя

Пароль превращаем в хэш; роль назначает сервер.

Сервис регистрации · фрагмент — Учебный пример

```
user = User(
    email=normalized_email,

hashed_password=hash_password(password),
    is_admin=False,
)
db.add(user)
# commit; при UNIQUE-конфликте rollback
```

Проверка	Ответ
Новые email/password	201
Тот же нормализованный email	409
Невалидный email / короткий пароль	422
В JSON добавили is_admin=true	422

UserRead: id, email, is_admin. Пароля и хэша в ответе нет.

↳ UNIQUE в базе закрывает гонку регистрации. Предварительного SELECT для этого недостаточно.

Шаг 2. Пароль проверяем через verify

Два хэша одного пароля могут отличаться: в них используется разная соль.

passlib · проверка, а не сравнение хэшей — Учебный пример

```
saved = hash_password(password)
verify_password(password, saved)  # True
verify_password("wrong", saved)   # False

# hash_password(password) == saved
# так пароль не проверяем
```

Что хранится

bcrypt-хэш в hashed_password.
Открытый пароль не сохраняется.

Граница bcrypt

Не более 72 байт UTF-8.
Число символов и число байт различаются.

↳ Хэш нужен для проверки пароля на входе. JWT нужен для последующих запросов.

Шаг 3. После логина выдаём JWT

Токен подтверждает результат входа на ограниченное время.

Claims · пример структуры, не живой токен — Учебный пример

```
{
  "sub": "7",
  "exp": "время окончания: +15 минут",
  "iss": "sneaker-practice",
  "aud": "sneaker-api"
}
```

Подпись

python-jose · HS256.
JWT_SECRET берём из .env.

Чего внутри нет

Пароля и hashed_password.
Роль читаем из актуальной БД.

↳ JWT подписан, но payload не зашифрован. Подпись и срок проверяются сервером при каждом запросе.

Шаг 4. Получить проверенного пользователя

Каждый защищённый запрос проходит одну и ту же цепочку.

01
Bearer
Извлечь токен

→

02
JWT
Подпись + срок + claims

→

03
User
Найти sub в БД

Псевдокод get_current_user — Учебный пример

```
нет токена / неверный JWT → 401
недопустимый sub → 401
пользователь не найден → 401
иначе вернуть User
```

/auth/me

Возвращает id, email, is_admin.
Хэш остаётся на сервере.

↳ Действительный формат токена не равен действительной подписи. Известный ID не равен входу.

Шаг 5. Выполнить вход в Swagger

Для JSON-логина сначала получаем токен, затем задаём его для следующих запросов.

1	**POST /auth/register**	Try it out → JSON email/password → Execute → 201
2	**POST /auth/login**	Те же данные → 200 → скопировать access_token
3	**Authorize**	В поле HTTPBearer вставить только токен → Authorize
4	**GET /auth/me**	Execute → свой email; после Logout → 401

↳ Swagger сам добавит Authorization: Bearer … . Пароль не отправляется вместо токена.

Шаг 6. Назначить учебного администратора

Права повышает локальный оператор для явно выбранной существующей учётной записи.

После регистрации этого аккаунта — Учебный пример

```
python -m app.promote_admin \
  admin@example.com

# Затем GET /auth/me
# is_admin: true
```

До команды

Обычный аккаунт: is_admin=false.
Запись каталога → 403.

После команды

Тот же JWT, роль читается из БД.
Запись становится доступна.

↳ Публичная регистрация не принимает роль. CLI работает с конфигурацией выбранной учебной базы.

Шаг 7. Защитить все три операции записи

Подключаем готовые сервисы M2 через общую зависимость администратора.

routers/catalog_write.py · фрагмент — Учебный пример

```
router = APIRouter(
    dependencies=[Depends(require_admin)]
)

# Три обработчика в этом роутере:
# POST   → create_product
# PUT    → update_product
# DELETE → delete_product
```

Порядок проверки

get_current_user → require_admin
→ обработчик → сервис записи

GET остаются публичными

Защищаем роутер изменения,
а не весь каталог целиком.

↳ Проверка роли должна действовать на POST, PUT и DELETE. Одной защиты кнопки или POST недостаточно.

Проверяем матрицу доступа

Для каждого участника отправлены POST, PUT и DELETE с корректными параметрами.

Кто	POST	PUT	DELETE
Без токена	401	401	401
Неверный токен	401	401	401
Пользователь	403	403	403
Администратор	201	200	204

↳ Для отказа проверяем и HTTP-код, и отсутствие изменений. Для успеха — фактическую запись.

14 Практика M3 · вход и права

Ревью M3 завершает защиту каталога

Проверяем вход, роли и отложенное условие M2. Затем готовимся к личной корзине.

401 · вход не подтверждён

Нет действительных данных входа.
Сервис записи не выполняется.

403 · прав не хватает

Пользователь известен, роль не подходит.
Данные каталога сохраняются.

↳ Сохраняем код, команды и результаты матрицы. M2 и M3 приняты; корзину реализуем в M4.